

Unified FPGA Design of Kyber and Dilithium with Provable Fault Tolerance

Siddhartha Chowdhury, Nimish Mishra, Sarani Bhattacharya, Debdeep Mukhopadhyay

Department of Computer Science and Engineering

Indian Institute of Technology Kharagpur, West Bengal, India

siddhartha.chowdhury92@gmail.com, neelam.nimish@gmail.com,

bhattacharya.sarani.iitkgp@gmail.com, debdeepmukhopadhyay@gmail.com

Abstract—Efficient and secure hardware implementations of post-quantum cryptographic schemes are critical for real-world adoption. In this work, we propose a unified FPGA-based architecture for Kyber and Dilithium that combines flexibility, lightweight design, and fault tolerance. The architecture adopts a microcoded, programmable datapath supporting both schemes with minimal area overhead, enabling seamless integration of modules such as SHAKE, sampling, and coefficient rounding. To enhance resilience against propagation-based fault attacks—which exploit effective/ineffective fault behavior in public-domain computations—we embed a probabilistic verification mechanism using rejection sampling. This countermeasure transforms deterministic operations into cryptographically constrained probabilistic processes that remain efficient under normal conditions while significantly degrading under adversarial faults. The result is a robust and compact design that not only supports both a lattice-based KEM and signature scheme, but also provides the first unified fault countermeasure architecture for Kyber and Dilithium, maintaining low retry counts and minimal performance degradation in fault-free environments.

Index Terms—FPGA, Kyber, Dilithium, Fault-Tolerance.

I. INTRODUCTION

Modern public-key cryptography (PKC) relies on hard problems like integer factorization and discrete logarithms. However, Shor’s algorithm makes these problems efficiently solvable on quantum computers, rendering traditional PKC insecure. In response, the cryptographic community, through the NIST standardization process, has advanced post-quantum cryptography (PQC), with lattice-based schemes emerging as leading candidates due to their worst-case hardness guarantees and efficient implementations. Notably, five of NIST’s seven final candidates are based on structured lattices.

Despite strong theoretical foundations, deploying PQC in constrained and performance-critical environments—such as automotive, industrial control, and embedded systems—remains challenging. Practical implementations must balance lightweight design, execution efficiency, and algorithmic agility. Area and energy constraints demand compact designs; real-time applications require speed; and evolving standards necessitate support for multiple schemes with minimal redesign effort.

Our earlier work introduced a unified, lightweight hardware architecture supporting Kyber (KEM) and Dilithium (signature)—two lattice-based NIST finalists. Their structural similarities enabled shared datapaths, leading to a microcoded,

programmable design supporting multiple parameter sets. Key components like sampling, polynomial arithmetic, and rounding were logic-shared where possible. A flexible polynomial multiplier achieved excellent area-time trade-offs. The architecture, prototyped on a NEXYS 4 DDR FPGA, outperformed existing designs in both resource efficiency and throughput.

While efficient, such architectures remain vulnerable to physical attacks—especially fault attacks. Recent studies have shown that single-bit leakages from faults in Kyber and Dilithium can recover sensitive data via fault propagation, often without effective hardware countermeasures.

To address this, we extend our prior architecture with a cryptographically grounded rejection sampling mechanism that detects and halts faulty operations. By introducing probabilistic checks on otherwise deterministic processes (e.g., sampling, rounding), our design ensures fault-induced deviations trigger high-probability rejections. For correct executions, the retry count remains low (typically 2–3), keeping performance within acceptable bounds.

We adopt the rejection sampling approach from [MM24], adapting it for lightweight hardware. Lookup-based comparators verify whether sampled or decoded values lie within valid statistical bounds, mitigating faults at key stages like `poly_tomsg` in Kyber and `y-generation` in Dilithium. Integrated into the datapath using compact tables, this logic provides reliable detection at minimal cost and energy overhead.

Our design is, to the best of our knowledge, the first unified, programmable, and fault-tolerant hardware architecture for both KEM and signature lattice-based PQC. It generalizes to other deterministic public-domain algorithms, enabling wide applicability without ad hoc countermeasures. The rejection logic integrates cleanly with our microcoded base, preserving agility and efficiency.

In summary, our contributions are:

- 1) A unified, microcoded, and fault-tolerant hardware architecture for Kyber and Dilithium PQC acceleration in constrained environments.
- 2) A novel probabilistic execution framework using rejection sampling to resist propagation-based fault attacks.
- 3) Integration of the countermeasure into our prior design with minimal area overhead and constant-time unfaulted execution.

- 4) FPGA-based validation demonstrating improved fault resilience alongside high efficiency and flexibility.

II. PRELIMINARIES

In this subsection, we describe the post-quantum primitives and the basic components used in this work.

A. Kyber

Kyber is an IND-CCA secure key encapsulation mechanism (KEM) based on the Module-LWE problem. It has three main algorithms: key generation, encryption, and decryption. Key parameters include the polynomial degree $n = 256$, prime modulus $q = 3329$, binomial distribution parameters η_1, η_2 , and an integer k defining the dimension $k \cdot n$ of the LWE problem. Security levels Kyber512, Kyber768, and Kyber1024 differ by $k = 2, 3, 4$, $\eta_1 = 3, 2, 2$, and $\eta_2 = 2, 2, 2$.

Key Generation:

Starting from a 32-byte seed, the algorithm generates a $k \times k$ matrix A with ring polynomials in $\mathbb{R}_q$. Secret and error polynomial vectors $\mathbf{s}, \mathbf{e}$ are sampled from a centered binomial distribution. The public key polynomial vector is

$$\mathbf{t} = A\mathbf{s} + \mathbf{e}.$$

The public key is $(\mathbf{t}, \rho)$, where ρ is a random seed, and the secret key encodes $\mathbf{s}$ along with hashes of the public key and seed.

Key Encapsulation:

Given a message m and public key, the algorithm regenerates A and samples random polynomials $\mathbf{r}, \mathbf{e}_1, \mathbf{e}_2$ from the binomial distribution. It computes

$$\mathbf{u} = A\mathbf{r} + \mathbf{e}_1, \quad v = \mathbf{t}^\top \mathbf{r} + e_2 + m.$$

These are compressed into ciphertext components c_1, c_2 and concatenated as ciphertext c , used to derive the shared secret.

Key Decapsulation:

Using ciphertext c and secret key $\mathbf{s}$, decompress $\mathbf{u}, v$, then recover

$$m' = v - \mathbf{s}^\top \mathbf{u}.$$

The decapsulation checks ciphertext validity by re-encrypting and comparing outputs, finally producing the shared secret.

B. Dilithium

Dilithium is a signature scheme based on Lyubashevsky's Fiat-Shamir with aborts framework and hard problems in module lattices. It consists of three algorithms: key generation, signing, and verification. The main parameters include polynomial degree $n = 256$, prime modulus $q = 8380417$, and public matrix dimensions k and l . Dilithium defines three security levels with parameter pairs $(k, l) = (4, 4), (6, 5), (8, 7)$.

Key Generation:

The algorithm generates a $k \times l$ matrix A with polynomials in $\mathbb{R}_q$. Secret vectors $\mathbf{s}_1, \mathbf{s}_2$ with coefficients in $[-\eta, \eta]$ are sampled. The public key's second part is computed as

$$t = A\mathbf{s}_1 + \mathbf{s}_2,$$

all over $\mathbb{R}_q$. To reduce public key size, A is replaced by a seed ρ that deterministically generates A . Additionally, the lower d bits of coefficients in t are stored in the secret key instead of the public key.

Signature Generation:

A masking vector y with coefficients in $[-\gamma_1, \gamma_1]$ is sampled to compute

$$w = Ay,$$

which is rounded as

$$w = w_1 \cdot 2\gamma_2 + w_0,$$

with $|w_0| \leq \gamma_2$. The challenge polynomial c , sparse with τ coefficients in $\{-1, 1\}$, is hashed from the message and w_1 . The candidate signature is

$$z = y + cs_1.$$

Rejection sampling prevents secret leakage by repeating the process if checks fail. Since only the upper bits of t are public, carry bits ("hints") h are computed to aid verification. The signature returned is (c, z, h) .

Signature Verification:

The verifier computes

$$w'_1 = \text{round}(Az - ct),$$

using hints h to compensate for missing lower bits of t . The challenge c is recomputed from the message and w'_1 and compared with the signature's c . Additionally, z is checked for norm validity consistent with signing constraints.

C. Propagation Based Fault Attacks

Many fault attacks on KEMs [KCS⁺24], [HMS⁺23], [PP21], [HPP21], [Del21], [FJKK⁺22] follow a common pattern: ① inject a fault during the decapsulation phase, and ② observe whether the derived key κ from the decapsulation phase results in a decryption failure. A failure (Line 8) indicates an effective fault, otherwise (Line 6) it is ineffective. The primary difference across these works lies in the fault model.

We briefly describe the attack in [PP21] on Kyber¹, illustrated in Fig. 1. During decryption, ciphertexts for plaintext bit 0 are centered around 0, while those for bit 1 are near $\frac{q}{2}$. The decoding involves: ① multiply by 2, ② add $\frac{q}{2}$, ③ extract the LSB.

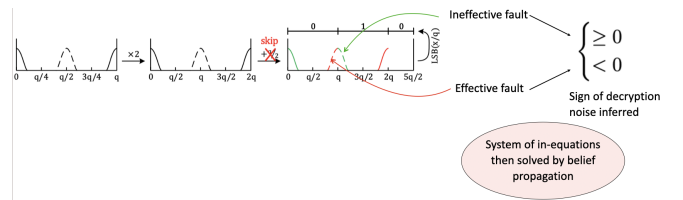


Fig. 1: Fault attack on Kyber from [PP21].

¹A NIST Round-3 lattice-based KEM finalist.

An instruction skip omits step 2, shifting the distribution left by $\frac{q}{2}$, which affects decoding. Based on success or failure, the attacker infers:

$$\langle \mathbf{r}[i], \mathbf{e} \rangle - \langle \mathbf{e}_1[i], \mathbf{s} \rangle + \mathbf{e}_2[i] \begin{cases} \geq 0, & \text{ineffective} \\ < 0, & \text{effective} \end{cases}$$

Here, $\mathbf{r}$, $\mathbf{e}$, $\mathbf{e}_1$, $\mathbf{e}_2$ are polynomials from the key generation phase, i is the faulted coefficient, and $\langle \cdot \rangle$ denotes dot product. Knowing $\mathbf{r}$, $\mathbf{e}_1$, and $\mathbf{e}_2$, the attacker builds inequalities over multiple faults to recover the secret $\mathbf{s}$.

Similar strategies apply to Masked Kyber [KCS⁺24] and NewHope [PAA⁺19], omitted here for brevity.

D. Rejection Sampling in Lattice-based Cryptography

Rejection sampling [VN63] samples from a target distribution f using a source g , and is key to lattice-based signatures [Lyu12], [DDLL13], including Dilithium [LDK⁺20], [DKL⁺18]. We restate the core result from [Lyu12]:

Lemma 1 (Rejection Sampling). *Let V be a set, $h : V \rightarrow \mathbb{R}$, $f : \mathbb{Z}^m \rightarrow \mathbb{R}$, and $\{g_v\}$ be distributions over $\mathbb{Z}^m$ such that $M \cdot g_v(\mathbf{x}) \geq f(\mathbf{x})$ for all $v \in V$, $\mathbf{x} \in \mathbb{Z}^m$. Then, the following two are statistically close:*

- 1) $v \leftarrow h$, $\mathbf{x} \leftarrow g_v$, output $(\mathbf{x}, v)$ with prob. $\frac{f(\mathbf{x})}{M g_v(\mathbf{x})}$
- 2) $v \leftarrow h$, $\mathbf{x} \leftarrow f$, output $(\mathbf{x}, v)$ with prob. $\frac{1}{M}$

ver

Note: Rejection sampling ensures $\mathbf{x}$ is statistically independent of v , which is often treated as secret material.

E. Keeping it Agile

Apart from optimizing the area-time product, agility² is crucial for automotive and cryptographic systems. Our architecture achieves this via a microcoded framework, enabling protocol and security-level reconfiguration. The datapath is composed of discrete compute units controlled by 106-bit instructions (40 bits for control, 66 for operand addresses). Custom instruction classes include:

Polynomial Arithmetic: Configure the NTT multiplier and control addition, subtraction, and point-wise multiplication.

Keccak Operations: Manage SHA3/SHAKE phases via the Keccak wrapper.

Read/Write: Govern data transfers between compute blocks and memory.

Kyber Functions: Encode/Decode as per [21a].

Dilithium Functions: Power2Round, MakeHint, UseHint, etc., as per [21b].

Arithmetic units are reused across security levels by updating instruction sequences and operand addresses (e.g., 896 instructions for Kyber- $k = 2$, 1344 for $k = 3$). Switching between Kyber and Dilithium is managed via instruction-controlled prime selection using multiplexers.

Unlike our design, the Saber-focused architecture in [AMJ⁺21] lacks such agility, supporting only LightSaber due to its fixed lifted prime, and cannot scale to higher security levels like FireSaber.

III. PROPOSED ARCHITECTURE AND DESIGN DECISIONS

A unique feature of the proposed architecture is a unified NTT-based polynomial multiplier coupled with a specially integrated Keccak core that is commonly available to both Kyber and Dilithium execution. The Keccak core is capable of computing both SHAKE and SHA3, and is primarily used for both pseudo-random number generation and hash computation. Sharing these two compute-intensive modules responsible for core processing tasks require a combined and highly optimized co-processor architecture. Additionally, there are a few auxiliary blocks that are further optimized to accelerate both Kyber and Dilithium. In this section, we shall elaborate design details and strategies of the combined architecture.

A. High Level Overview of the Shared Architecture of Kyber and Dilithium

The complete hardware architecture of the proposed design is shown in Figure 2, illustrating the combined datapath for Kyber and Dilithium along with instruction and data memory. Instructions stored in the instruction memory control the execution sequence for either scheme. The datapath integrates logic and functional modules necessary for both.

Figure 2 highlights the unified NTT multiplier and Keccak core, shared across Kyber and Dilithium. This sharing significantly reduces the area-time product and offers speedups over software implementations, as hardware modules like **Encode** and **Decode** (rewiring operations) incur negligible latency.

Dilithium's **Power2Round** module splits a value $r \in \mathbb{Z}_q$ as $r = r_1 \cdot 2^d + r_0$, where $r_0 = r \bmod \pm 2^d$ and $r_1 = (r - r_0)/2^d$, using shifts and bit extractions in hardware. Modules like **Decompose**, **MakeHint**, and **UseHint** use lightweight combinational logic for constant-based add/subtract/multiply operations, depending on the security level. Their outputs are routed through multiplexers, selecting values based on control signals. Further details on the unified NTT and Keccak blocks are presented in subsequent sections.

B. Design of Unified NTT Multiplier

Polynomial multiplication is one of the most computationally intensive tasks in both Kyber and Dilithium.

To unify their multiplier architecture, we exploit the structure of their field primes. Although both use NTT-friendly primes, Kyber's prime has only a 256'th root of unity, while Dilithium's has both 256'th and 512'th roots, supporting 7 and 8 levels of NTT respectively. We use incomplete NTT for both, producing $n/2$ degree-1 polynomials. Multiplication of these degree-1 polynomials occurs in the ring $\mathbb{Z}_q[x]/(x^2 - \omega^i)$, where ω is the n -th root of unity and i depends on the coefficient index. These polynomials cannot be multiplied coefficient-wise. Below, we explain how we combined Kyber and Dilithium polynomial multiplication on common ground.

²<https://semiengineering.com/the-battle-for-post-quantum-security-will-be-won-by-agility/>

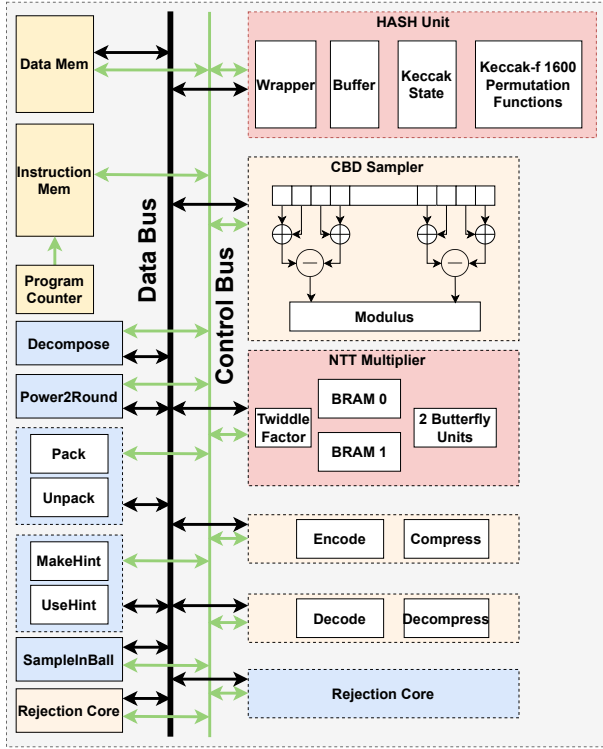


Fig. 2: Hardware architecture of the crypto co-processor

Unification Based on Root of Unity. We use the 256'th root of unity for both schemes, reducing Dilithium's NTT levels from 8 to 7 and lowering total multiplications. For point-wise multiplication at seven NTT levels, we adopt Karatsuba's method as in [XL21]. The basecase multiplication (from Kyber specification [21a]) is:

$$h_{2i} + h_{2i+1}X = (f_{2i} + f_{2i+1}X) \cdot (g_{2i} + g_{2i+1}X) \mod (X^2 - \zeta^{2br(i)+1})$$

Naively, this requires 5 multiplications, but Karatsuba reduces it to 4 as shown in Equation 1.

$$\begin{aligned} h_{2i} &= f_{2i}g_{2i} + f_{2i+1}g_{2i+1} \cdot \zeta^{2br(i)+1} \\ h_{2i+1} &= (f_{2i} + f_{2i+1})(g_{2i} + g_{2i+1}) - (f_{2i}g_{2i} + f_{2i+1}g_{2i+1}) \end{aligned} \quad (1)$$

Multiplying two Dilithium polynomials using the 512'th root of unity requires 3328 multiplications, while the Karatsuba method with the 256'th root requires only 3200. This reduces the overall latency of scheme execution.

Reduction Architecture Based on Prime Structure. Kyber's prime is of the form $q = k \cdot 2^m + 1$ with $k = 13$ and $m = 8$ ($k < 2^m$). Prior Kyber hardware [BNAMK21] uses this structure for KRED2 reduction. We observe that Dilithium's prime shares this form with $k = 1023$ and $m = 13$ and implement KRED2 as in Algorithm 1. Exploiting this, we apply KRED2 reduction to both Kyber and Dilithium. The

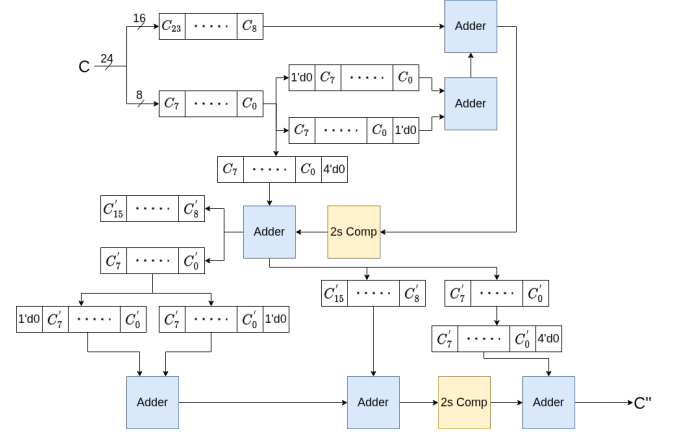


Fig. 3: Reduction Circuit of Kyber

KRED2 algorithm takes input integer c and outputs D such that $D \equiv k^2 \cdot C \pmod{q}$.

Algorithm 1 KRED2 for Dilithium

Require: A binary number $C = (c_{45}, \dots, c_0)_2$, $k = 1023$, $m = 13$, $q = 8380417 = k \cdot 2^m + 1$

Ensure: $C' = k^2 \cdot C \pmod{q}$

$$C_l = (c_{12}, \dots, c_0)_2$$

$$C_h = (c_{45}, \dots, c_{13})_2$$

$$C' = k \cdot C_l - C_h$$

$$C'_l = (c'_{12}, \dots, c'_0)_2$$

$$C'_h = (c'_{33}, \dots, c'_8)_2$$

$$C' = k \cdot C'_l - C'_h$$

KRED2 Algorithm. The KRED2 algorithm [BNAMK21] implementing the reduction is given in Algorithm 1. The extra factor k^2 can be removed by incorporating k^{-2} into the stored twiddle factors. To optimize modulo operations, we decompose k as $(16 - 3)$ for Kyber and $(1024 - 1)$ for Dilithium. Thus, C' is expressed as $C' = ((C_l \ll 4) - (3 \cdot C_l + C_h))$ for Kyber and $C' = ((C_l \ll 10) - (C_l + C_h))$ for Dilithium, realizable with combinational logic. Our modular reduction unit includes these combinational blocks followed by a multiplexer selecting output for Kyber or Dilithium, reducing latency at a small area cost. The reduction circuit for Kyber is shown in Figure 3.

Although Dilithium's prime supports 512-th roots of unity, we use the 256-th root to perform only 7 NTT levels, lowering multiplication counts with Karatsuba (details later). We implement NTT multipliers using Cooley-Tukey for forward NTT and Gentleman-Sande for inverse NTT, avoiding bit-reversal. This approach, common in prior works, applies Decimation in Time and Decimation in Frequency, respectively.

NTT Computation. We use Cooley-Tukey (*Decimation in Time*) for forward NTT and Gentleman-Sande (*Decimation in Frequency*) for reverse NTT to avoid bit-reversal. A similar design was proposed in [XL21], but our fast reduction technique achieves significantly lower latency. Figure 4 depicts the butterfly unit processing two coefficients simultaneously.

The NTT multiplier includes 2 butterfly units, processing 4 polynomial coefficients per iteration. It uses two

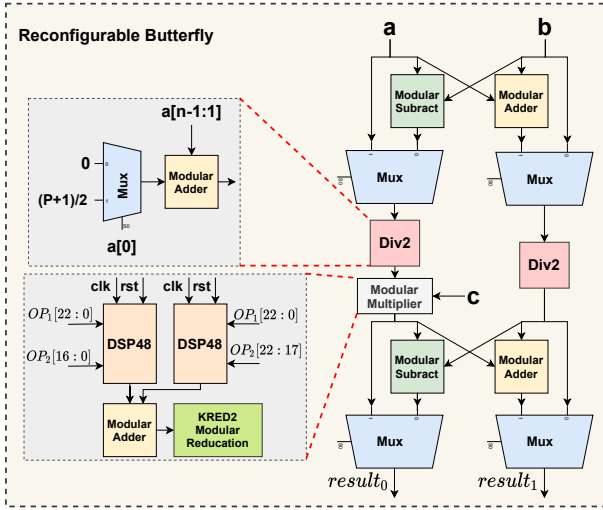


Fig. 4: Re-configurable butterfly unit

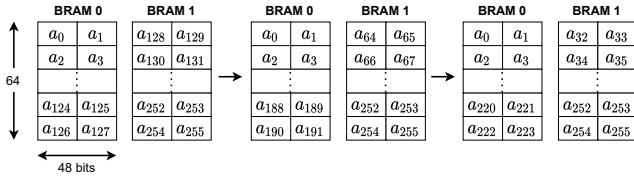


Fig. 5: NTT multiplier scheduling in the NTT RAM

BRAMs storing pairs of coefficients indexed by $s \in \{128, 64, 32, 16, 8, 4, 2\}$ to feed the butterflies. Moreover, the twiddle factors and the address for the coefficients are stored in Block-ROMs. Thus, each unit receives coefficients j , $j + s$, $j + 1$, and $j + s + 1$ simultaneously for $j \in [0, 255]$. Figure 5 shows the execution schedule with coefficients a_i . Both forward and reverse NTT take 448 clock cycles.

The butterfly unit in Figure 4 operates in three modes: Forward NTT, Inverse NTT, and point-wise multiplication, controlled by MUX select lines. Input c switches between twiddle factors or polynomial coefficients based on mode. These are governed by polynomial multiplier instructions (Section II-E), enabling agility across security levels and schemes. Our fast reduction technique further reduces the design’s area-time product.

C. Sampling

Kyber Sampler. Kyber employs a binomial sampler to draw integers from a centered binomial distribution. We implement two samplers, $\mathcal{S}_1$ and $\mathcal{S}_2$, for $\eta = 2$ and $\eta = 3$, respectively. $\mathcal{S}_1$ uses 16 parallel adder cores to generate and buffer 16 samples per cycle, while $\mathcal{S}_2$ processes three 64-bit input words to yield 32 two-byte samples across cycles. Instruction controls (Section II-E) enable dynamic selection based on the required security level.

Dilithium Sampler. Dilithium employs three rejection sampling cores, directly fed by the KECCAK unit using a lookup-based approach. Instruction-level control (Section II-E) man-

ages core selection and operation. Both matrix-vector and secret key coefficients are generated via uniform rejection sampling.

D. Programmable Keccak Core

The Keccak core [15] is used for hashing and pseudo-random number generation in both Kyber and Dilithium. Kyber operates in four Keccak modes—SHA3-256, SHA3-512, SHAKE-128, and SHAKE-256—while Dilithium uses SHAKE-128 and SHAKE-256. All modes are implemented using the Keccak sponge construction based on the Keccak-f[1600] permutation. A single core is reused across all modes using distinct wrappers and buffers, which are selected via micro-coded control signals, as discussed in Section II-E.

Mode selection defines the rate r and the domain-specific padding suffix. Wrappers extract appropriate r -bit blocks from the 1600-bit state, while the FSM and instruction memory manage the absorb and squeeze phases. The core continues to absorb input until sufficient data is available, and then transitions to hashing using the *finish* signal.

This unified Keccak design directly feeds the NTT and rejection sampling modules, reducing communication latency and saving resources. Although the Keccak unit consumes 10879 LUTs on Artix-7, the multiplexed architecture supports all security levels of both Kyber and Dilithium efficiently. The full Keccak-f[1600] requires 48 clock cycles per hash operation.

E. Modifications to the Architecture for Fault Attack Countermeasure

In our architecture, we implement lightweight fault-tolerance mechanisms for both Kyber and Dilithium by leveraging the rejection sampling framework proposed in [MM24]. This framework introduces a principled approach to prevent fault propagation attacks by enforcing probabilistic verification of input distributions at vulnerable stages of cryptographic computations.

For Kyber, we apply this methodology during the `poly_tomsg` routine in the decryption phase, which is a known target for propagation-based fault attacks. We design a hardware comparator that cross-verifies the decoded message coefficients against a precomputed lookup table of valid output values derived from the expected distribution of correctly decoded messages. These allowed values for the decoded output were emulated from the valid distribution described in [MM24], ensuring that only correctly distributed message values proceed through the cryptographic flow, effectively blocking fault-induced biases.

For Dilithium, we utilize a similar principle during the generation of the intermediate vector y , which plays a crucial role in the signing operation. Following [LDK⁺20], the coefficients of y are sampled from a discrete distribution defined as

$$\mathcal{G} = \{q + \gamma_1 - 1 - v_j \mid v_j \in \{0, \dots, 2\gamma_1 - 2\}, j \in \{0, \dots, 255\}\},$$

where q and γ_1 are constants in Dilithium’s parameterization. This “safe” set $\mathcal{G}$ represents all coefficients of y that effectively

hide the secret-dependent information in $z = y + sc$. We emulate the allowed values of y from this set as described in [MM24] and implement a simple hardware comparator to enforce that only coefficients within this safe domain are accepted, while all others are rejected. This prevents leakage of secret-dependent information under faulted conditions.

This results in a fault-tolerant yet efficient design, as the hardware cost of the comparator logic is minimal, and the probabilistic nature of the mechanism aligns well with the original protocol semantics. Overall, our fault-tolerance architecture, guided by the rejection sampling framework of [MM24], provides robust protection against a broad class of active physical attacks with negligible area and performance overhead.

F. Auxiliary building blocks

Alongside core functional blocks, the design includes auxiliary modules for optimized computation. The data memory outputs two coefficients per cycle. **Pack** and **Unpack** modules efficiently handle rewiring and basic arithmetic. Modules like **Power2Round**, **MakeHint**, **UseHint**, **Encoder**, **Decoder**, and **Decomposer** implement constant-based combinational logic tailored to Kyber and Dilithium [21a], [21b]. The polynomial arithmetic unit supports non-blocking add/subtract operations for parallelism with the NTT.

1) *Polynomial Compression and Decompression*: Kyber employs polynomial compression to reduce coefficient bit-widths d for efficient transmission, using the mapping:

$$\text{Compress}_q(x, d) = \left\lfloor \frac{2^d \cdot x + \frac{q}{2}}{q} \right\rfloor \bmod 2^d,$$

where $q = 3329$ and $x \in \mathbb{Z}_q$ is first mapped to a positive representative via $x \leftarrow x + ((x \gg 15) \wedge q)$. To avoid costly division in hardware, fixed-point arithmetic is used with precomputed constants. For example, when $d = 4$, the compressed output is:

$$t = (((x \ll 4) + 1665) \cdot 80635) \gg 28 \bmod 16,$$

where 1665 approximates $q/2$ and 80635 is a scaling factor. For $d = 1$ (used in `poly_tomsg`), this simplifies to a single-bit output. A combinational circuit computes one d -bit compressed coefficient per cycle using shift-and-add logic, enabling efficient byte packing (e.g., $r[i] = t_{2i} \mid (t_{2i+1} \ll 4)$ for $d = 4$).

Decompression restores coefficients using:

$$r = \left\lfloor \frac{a \cdot q + 2^{d-1}}{2^d} \right\rfloor,$$

where $a \in [0, 2^d - 1]$. For $d = 4$, two 4-bit values are extracted from each byte and scaled; for $d = 5$, 5-bit groups are scaled similarly; and for $d = 1$, each bit $a \in \{0, 1\}$ maps to $\lfloor \frac{a \cdot q + 1}{2} \rfloor$. Hardware decompression uses masking, shifting, fixed multipliers or shift-add structures, and supports parallel units for high-throughput Kyber implementations.

2) *Polynomial Serialize and Deserialize Unit*: In Kyber, each polynomial contains 256 coefficients modulo $q = 3329$, with each 12-bit coefficient compactly stored in a 384-byte format. Two coefficients are packed into three bytes: the first holds the lower 8 bits of the first coefficient, the second combines the upper 4 bits of the first and the lower 4 bits of the second, and the third holds the upper 8 bits of the second.

The serialization unit employs conditional adders and bit-wise logic under iterative control to efficiently pack coefficients in hardware, ensuring low area and high throughput.

Deserialization recovers two 12-bit coefficients from every three bytes using shift and mask operations:

$$\begin{aligned} t_0 &= a[3i] \mid (a[3i+1] \ll 8), & t_0 &= t_0 \& 0\text{xFFF} \\ t_1 &= (a[3i+1] \gg 4) \mid (a[3i+2] \ll 4), & t_1 &= t_1 \& 0\text{xFFF} \end{aligned}$$

This fully combinational, multiplier-free logic supports both streaming and parallel modes.

3) *Auxiliary Function in Dilithium*: Dilithium employs four auxiliary arithmetic functions—`power2round`, `decompose`, `make_hint`, and `use_hint`—to enable efficient rounding, compression, and error correction during signature generation and verification. These functions manipulate polynomial coefficients through simple bitwise operations and are lightweight by design.

The `power2round` function splits a coefficient a into a coarse part a_1 and a fine part a_0 , such that $a = a_1 \cdot 2^D + a_0$, enabling high-precision encoding. The `decompose` function performs a similar task with a different granularity, producing a_1 and a_0 such that $a = a_1 \cdot \text{ALPHA} + a_0$, with special care to handle symmetric boundaries and exceptional cases based on GAMMA_2 values, which vary across security levels.

The `make_hint` function generates a single-bit indicator denoting whether a_0 overflows into a_1 during rounding, while `use_hint` reconstructs or adjusts a_1 based on this hint during verification.

These functions can be efficiently implemented on FPGAs for all Dilithium security levels using only shifts, additions, and conditional logic. Furthermore, lookup tables (LUTs) can be employed for `decompose` and `use_hint`, where the limited range of valid a_1 values (e.g., ≤ 15 or 43) enables fast, precomputed access without complex control flow. This makes these functions ideal for pipelined and parallel hardware architectures.

G. Prototype for Proposed Architecture

We have chosen a lightweight Xilinx board NEXYS 4 DDR which houses an Artix-7 FPGA and a soft-core microprocessor Microblaze which are recently being promoted for automotive contexts³. In order to achieve a stand-alone design, we have applied the reset signal to our hardware design using the softcore Microblaze microprocessor available on our target platform. Figure 6 illustrates the handshake between the Microblaze and our crypto co-processor.

³https://www.xilinx.com/support/documentation/white_papers/wp501-microblaze.pdf

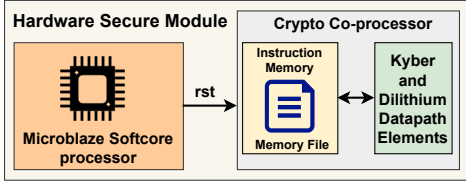


Fig. 6: Overall architecture of the prototype HSM

TABLE I: Total area utilization of the different components in the unified architecture

Algorithm	Components	LUTs	DSPs	BRAMs
Dilithium	Decompose	504	-	-
	MakeHint	80	-	-
	UseHint	708	-	-
	Powe2Round	75	-	-
	Pack	658	-	-
	Unpack	325	-	-
	Encode	354	-	-
	Decode	160	-	-
	SampleInBall	485	-	-
	Verify	16	-	-
	Rejection Core	718	-	-
Kyber	Compress/Decompress	258	-	-
	Encode	581	-	-
	Decode	268	-	-
	COPY	30	-	-
	CMOV	35	-	-
	Rejection Core	185	-	-
	Verification Core	98	-	-
Common	KECCAK	10879	-	-
	Data Memory	-	-	19
	NTT Multiplier	2899	4	5
	Controller	2618	-	5
Total		22125	4	29

IV. RESULTS AND COMPARISON

In this section, we evaluate the performance of our proposed architecture against state-of-the-art designs. It utilizes 22125 LUTs, 12531 FFs, 4 DSPs, and 29 BRAMs, achieving a maximum frequency of 161 MHz. Detailed resource usage and latency for Kyber and Dilithium are presented in Table III and Table II, based on the Artix-7 platform. Figure 7 and Table IV show a comparison of the area-time (AT) product ($\text{LUT} \times \text{latency} \times 10^{-6}$), where our design significantly outperforms existing combined Kyber-Dilithium architectures. Table I presents the component-wise area breakdown, highlighting KECCAK and the NTT multiplier as the most resource-intensive blocks. Moreover this is the first reported unified design of Kyber and Dilithium to achieve fault tolerance.

The design in [AMI⁺23] also integrates Kyber and Dilithium but uses a Zynq Ultrascale+ board, separate KECCAK and NTT cores, and lacks an application-focused perspective. In [GJCJ23], the design achieves very low resource consumption at the cost of high latency of execution. Our optimized Dilithium scheduling achieves lower latency with fewer resources. The authors of [ZZW⁺21] reduce latency using multiple butterfly units in a pipeline with optimised rejection sampling modules, increasing area; we outperform them in AT product on Artix-7.

In Kyber-specific designs, [19] focuses on RISC-V-based low-power implementations but incurs higher latency due to hardware/software co-design. Similarly, [XL21] presents a

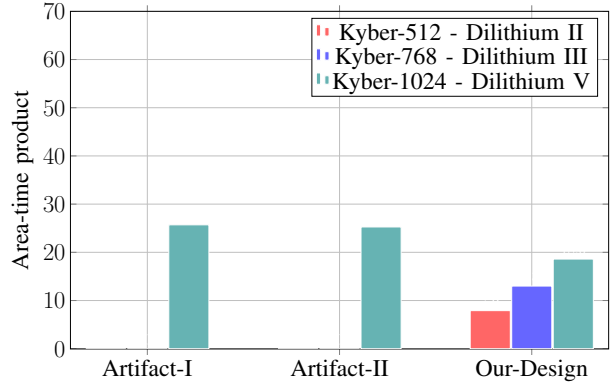


Fig. 7: Comparison of area-time product with existing designs

compact Kyber implementation, whereas our design achieves superior latency through a balanced high-speed architecture. While [GL23] demonstrates impressive latency and low resource usage through innovative conflict-free memory access, our unified architecture not only maintains a competitive area-delay product but also introduces fault tolerance—particularly when integrated with the state-of-the-art Dilithium hardware.

By combining Kyber from [GL23] with Dilithium from [GJCJ23] (Artifact I) or [ZZW⁺21] (Artifact II), and comparing the cumulative AT product, our design consistently performs better, as shown in Figure 7.

V. CONCLUSION

This paper presents a unified, microcoded, and fault-tolerant hardware architecture for Kyber and Dilithium. Extending our previous work, we incorporate a lightweight rejection sampling mechanism to counter fault attacks with minimal overhead. The design preserves agility across parameter sets while improving robustness and efficiency. FPGA validation confirms its suitability for secure, resource-constrained environments.

VI. ACKNOWLEDGEMENT

The authors would like to thank the Department of Science & Technology (DST), Government of India, and the Center for Development of Telematics (C-DOT) for partially funding this work through the project entitled “Development of Tamper-Proof SCA/FI-Resistant 10 Gbps Post-Quantum In-Line IP Network Encryptor, Post-Quantum TLS ASIC (PQ-TLS), and TLS Accelerator PCIe Card Using PQ-TLS ASIC for TLS Offloading in Servers,” under the “National Quantum Mission (NQM)” initiative and the “C-DOT STAR PhD Scholarship Program.”

REFERENCES

- [AMI⁺23] Aikata Aikata, Ahmet Can Mert, Malik Imran, Samuel Pagliarini, and Sujoy Sinha Roy. Kali: A crystal for post-quantum security using kyber and dilithium. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 70(2):747–758, 2023.

TABLE II: Comparison of resource utilization and timing details of Dilithium with existing architecture

Works	Algorithm	LUTs	FF	DSP	BRAM	Max Freq (MHz)	Keygen		Sign		Verify		AT Product	Fault Tolerant
							cycles	μs	cycles	μs	cycles	μs		
[ZZW ⁺ 21]	Dilithium II	29998	10366	10	11	96.9	4172	43.05	28091	289.89	4422	45.63	10	No
	Dilithium III	29998	10366	10	11	96.9	5851	60.38	44706	461.36	6181	63.78	15.7	No
	Dilithium V	29998	10366	10	11	96.9	8765	90.45	48996	505.63	9039	93.28	17.9	No
[GJCJ23]	Dilithium V	13975	6845	4	35	163	63.2K	387	113.9K	699	67.9K	416	15.5	No
	Dilithium II	22125	12531	4	29	161	16112	100	32168	200	18178	113	6.9	Yes
Our Design	Dilithium III	22125	12531	4	29	161	32102	199.4	56294	350	29084	181	11.7	Yes
	Dilithium V	22125	12531	4	29	161	49102	305	80420	500	43627	270	17.03	Yes

TABLE III: Comparison of resource utilization and timing details of Kyber with existing architecture

Works	Algorithm	LUTs	FF	DSP	BRAM	Max Freq (MHz)	Keygen		Encaps		Decaps		AT Product	Fault Tolerant
							cycles	μs	cycles	μs	cycles	μs		
[19]	Kyber-512	15K	3K	11	14	25	75K	3000	132K	5280	142K	5680	164.4	No
	Kyber-768	15K	3K	11	14	25	112K	4480	178K	7120	191K	7640	221.4	No
	Kyber-1024	15K	3K	11	14	25	149K	5960	223K	8920	241K	9640	278.4	No
[XL21]	Kyber-512	7K	5K	2	3	161	4K	24.8	5K	31.05	7K	43.47	0.5	No
	Kyber-768	7K	5K	2	3	161	6K	37.2	8K	49.68	10K	62.11	0.77	No
	Kyber-1024	7K	5K	2	3	161	9K	55.9	11K	68.32	14K	89.95	1.07	No
[GL23]	Kyber-512	7.3K	3.2K	4	5	200	2.4K	12.1	3K	15	4.2K	21.2	0.26	No
	Kyber-768	7.3K	3.2K	4	5	200	4.2K	21.2	5K	25.2	7K	34.9	0.43	no
	Kyber-1024	7.3K	3.2K	4	5	200	6.8K	34.1	8K	40.9	10.2K	51.1	0.67	No
Our Design	Kyber-512	22K	12K	4	29	161	2.5K	15.85	3.4K	21.27	4.2K	26.32	1.05	Yes
	Kyber-768	22K	12K	4	29	161	3.5K	21.9	4.3K	27.11	5.1K	31.85	1.304	Yes
	Kyber-1024	22K	12K	4	29	161	4.4K	27.5	5.3K	33.11	6.4K	39.89	1.61	Yes

TABLE IV: Comparison of Area-Time Product computation with existing designs

Artifacts	Algorithm	LUTs	Sign (μs)	Verify (μs)	Encaps (μs)	Decaps (μs)	AT Product
[ZZW ⁺ 21]+ [GL23]	Dilithium-V + Kyber-1024	37298	505.63	93.28	40.9	51.1	25.7
[GJCJ23]+ [GL23]	Dilithium-V + Kyber-1024	21275	699	416	40.9	51.1	25.67
Our Design	Dilithium-II + Kyber-512	22125	200	113	21.27	26.32	7.9
	Dilithium-III + Kyber-768	22125	350	181	27.11	31.85	13.0
	Dilithium-V + Kyber-1024	22125	500	270	33.11	39.89	18.6

[AMJ⁺21] Aikata Aikata, Ahmet Can Mert, David Jacquemin, Amitabh Das, Donald Matthews, Santosh Ghosh, and Sujoy Sinha Roy. A unified cryptoprocessor for lattice-based signature and key-exchange. *Cryptology ePrint Archive*, Paper 2021/1461, 2021.

[BNAMK21] Mojtaba Bisheh-Niasar, Reza Azarderakhsh, and Mehran Mozaffari-Kermani. High-speed NTT-based polynomial multiplication accelerator for CRYSTALS-kyber post-quantum cryptography. *Cryptology ePrint Archive*, Paper 2021/563, 2021.

[DDLL13] Léo Ducas, Alain Durmus, Tancrede Lepoint, and Vadim Lyubashevsky. Lattice signatures and bimodal gaussians. In *Annual Cryptology Conference*, pages 40–56. Springer, 2013.

[Del21] Jeroen Delvaux. Roulette: A diverse family of feasible fault attacks on masked kyber. *Cryptology ePrint Archive*, 2021.

[DKL⁺18] Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 238–268, 2018.

[FJKK⁺22] Michael Fahr Jr, Hunter Kippen, Andrew Kwong, Thinh Dang, Jacob Lichtinger, Dana Dachman-Soled, Daniel Genkin, Alexander Nelson, Ray Perlner, Arkady Yerukhimovich, et al. When frodo flips: End-to-end key recovery on frodokem via rowhammer. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 979–993, 2022.

[GJCJ23] Naina Gupta, Arpan Jati, Anupam Chattopadhyay, and Gautam Jha. Lightweight hardware accelerator for post-quantum digital signature crystals-dilithium. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 70(8):3234–3243, 2023.

[GL23] Wenbo Guo and Shuguo Li. Highly-efficient hardware architecture for crystals-kyber with a novel conflict-free memory access pattern. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 70(11):4505–4515, 2023.

[HMS⁺23] Julius Hermelink, Erik Mårtensson, Simona Samardjiska, Peter Pessl, and Gabi Dreier Rodosek. Belief propagation meets lattice

reduction: Security estimates for error-tolerant key recovery from decryption errors. *Cryptology ePrint Archive*, 2023.

[HPP21] Julius Hermelink, Peter Pessl, and Thomas Pöppelmann. Fault-enabled chosen-ciphertext attacks on kyber. In *Progress in Cryptology—INDOCRYPT 2021: 22nd International Conference on Cryptology in India, Jaipur, India, December 12–15, 2021, Proceedings 22*, pages 311–334. Springer, 2021.

[KCS⁺24] Suparna Kundu, Siddhartha Chowdhury, Sayandeep Saha, Angshuman Karmakar, Debdeep Mukhopadhyay, and Ingrid Verbauwhede. Carry your fault: A fault propagation attack on side-channel protected lwe-based kem. *arXiv preprint arXiv:2401.14098*, 2024.

[LDK⁺20] Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. Crystals-dilithium. *Algorithm Specifications and Supporting Documentation*, 2020.

[Lyu12] Vadim Lyubashevsky. Lattice signatures without trapdoors. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 738–755. Springer, 2012.

[MM24] Nimish Mishra and Debdeep Mukhopadhyay. Probabilistic algorithms with applications to countering fault attacks on lattice based post-quantum cryptography. *Cryptology ePrint Archive*, Paper 2024/551, 2024.

[PAA⁺19] T Pöppelmann, E Alkim, R Avanzi, J Bos, L Ducas, A de la Piedra, P Schwabe, D Stebila, MR Albrecht, E Orsini, et al. Newhope-submission to the nist post-quantum cryptography standardization project. *NIST National Institute of Standards and Technology*, 2019.

[PP21] Lukas Prokop and Peter Peßl. Fault attacks on cca-secure lattice kems. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(2):37–60, 2021.

[15] Guido Bertoni *et al.* Keccak. *IACR eprint*, 2015.

[19] Utsav Banerjee *et al.* Sapphire: A configurable crypto-

- processor for post-quantum lattice-based protocols (extended version). IACR eprint, 2019.
- [21a] Roberto Avanzi *et al.* Crystals-kyber. Proposal to NIST PQC ' Standardization, Round3, 2021.
- [21b] S. Bai *et al.* Crystals-dilithium. Proposal to NIST PQC ' Standardization, Round3, 2021.
- [VN63] John Von Neumann. Various techniques used in connection with random digits. *John von Neumann, Collected Works*, 5:768–770, 1963.
- [XL21] Yufei Xing and Shuguo Li. A compact hardware implementation of cca-secure key exchange mechanism crystals-kyber on fpga. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(2):328–356, Feb. 2021.
- [ZZW⁺21] Cankun Zhao, Neng Zhang, Hanning Wang, Bohan Yang, Wenping Zhu, Zhengdong Li, Min Zhu, Shouyi Yin, Shaojun Wei, and Leibo Liu. A compact and high-performance hardware architecture for crystals-dilithium. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(1):270–295, Nov. 2021.